

Reviewer Guide

[Platform Basics](#)[Env Setup Review](#)[Full Solution Review](#)[Edge cases & rulings](#)

Feedback and scoring

Giving feedback

Below every field in a task you'll see a speech bubble icon. Click it to open the feedback panel for that specific field. When adding feedback, select a type and write a clear explanation.

Feedback types

- **Approve** — The field meets all requirements
- **Request Changes** — The field needs fixes before it can be accepted
- **Comment** — General observation that doesn't block approval

Approve vs Request Changes

- Use **Approve** when the task meets quality standards across all fields
- Use **Request Changes** when there are issues the fellow needs to fix
- Always provide specific, actionable feedback explaining what needs to change

Writing good feedback

- **Be specific** — reference exact files, lines, or values that need fixing
- **Be actionable** — tell the fellow what to do, not just what's wrong
- **Be constructive** — explain why something matters

Scoring rubric

Rate each task from 1–5:

Score	Meaning	File Paths	Problem Statement	Test Case Coverage
5	Exceptional	All paths verified	Model-readable, no leakage, all requirements explicit	F2P tests cover the core bug/feature, P2P tests comprehensive

Score	Meaning	File Paths	Problem Statement	Test Case Coverage
4	Good	All paths exist, minor formatting issues	Valid and clear, missing one non-critical detail	F2P tests valid, P2P tests cover main cases
3	Acceptable	One path close but not exact	Understandable, one minor issue	F2P tests exist but miss an edge case
2	Below standard	Multiple paths missing	Contains Git/PR references or too vague	F2P tests trivial or don't target the actual fix
1	Unacceptable	Fabricated paths	Not a prompt at all — changelog or placeholder	No meaningful test coverage or tests unrelated to the task

Edit-in-place vs. send back

Edit-in-place (then approve with a Comment-type feedback note):

- Typos, whitespace, or formatting in `problem_statement`, `problem_statement_variant`, or `hints`
- File-path corrections in `FAIL_TO_PASS` / `PASS_TO_PASS` when the intended file is unambiguous
- A missing trivial field whose value is clearly derivable (e.g. `language` obvious from the repo)
- Removing template-padding headers (`## Problem`, etc.) when the prose underneath is fine
- Rewriting `problem_statement` to fix vagueness, when the intent is clear from the `[sol]` commit and PR body
- Removing leakage from `problem_statement` (PR/git references, "the original fix", "see #123") — rewrite as a clean prompt
- Tightening `problem_statement_variant` — voice shift, length adjustment, removing symbol names that leak from the main statement
- Removing implementation leakage from hints — leave the signposts (file paths, function names, error messages); strip step-by-step fix descriptions

Always send back — never edit in place:

- Weak F2P design (symbol-existence tests, missing behaviors, "test_solution_applied" patterns, F2P scope mismatched with `problem_statement` intent)
- Validity issues (LOC, no tests, dep-bump-only, force-pushed history, modifications to `main`)
- Solution code incomplete or incorrect
- Commit-shape issues (missing `[sol]` / `[f2p]` , mixed code+tests, extra commits, wrong prefixes)
- Docker-PR scope issues (changes outside the six allowed paths)

Edit and approve for prose-level fixes

If the only issue with a submission is prose-level (vague wording, leakage, weak hints), edit and approve instead of sending back. The fellow gets through faster and you spend less time re-reviewing the same task.

If you edit, you must also mirror

When you edit `problem_statement` or `problem_statement_variant` in `.helix/metadata.json`, the HAI platform copy must also be updated. Otherwise the submission check will reject with 'problem_statement mismatch'. Update both before approving.

Decision heuristic

Edit in place if the fix is to the WORDS. Send back if the fix is to the WORK — the test design, the solution, the commit history. The fellow learns nothing from a typo correction; they learn a lot from rebuilding a weak F2P set.

[Previous](#)[Next](#)